

- i. Introduction: AI for Science
 - 1.1. Physics-driven methods
 - 1.2. Data-driven methods
- ii. 基本框架
 - 2.1. Encoder -Decoder 框架
 - 2.2. 计算过程
 - 2.3. 算子学习的分辨率不变性
- iii. DeepONet
 - 3.1. 架构设计
 - 3.2. 理论保证
- iv. PCA-Net
 - 4.1. 基本框架
 - 4.2. 训练方法
 - 4.3. 理论分析
- v. FNO
 - 5.1. 基本框架
 - 5.2. 训练过程
 - 5.3. 理论保证
- vi. 误差分解

Operator learning: A theoretical perspective

1. Introduction: AI for Science

算子学习是指将数据驱动式学习（通常是非线性的）算子，即从数据中寻找巴拿赫空间之间映射的逼近方法。这些算子通常在函数的Banach Space之间进行映射。一个常见的场景即为解 PDEs。目前机器学习应用于科学计算的主流算法主要分两类，主要是基于物理驱动和数据驱动的方法。这些方法在不同的应用场景中展现出各自的优势和局限性。

1.1. Physics-driven methods

物理驱动学习遵循无监督学习范式，其主要思想是通过物理定律来约束模型的学习过程。典型的算法包括 Physics-Informed Neural Network (PINNs), Deep Ritz Methods(DRMs) 以及各种变体。

PINN 的基本思想是将物理定律嵌入到神经网络的训练过程中，通过最小化损失函数来学习 PDE 的解。具体来说，PINN 通过引入物理约束，将 PDE 的残差作为损失函数的一部分，使得网络在训练过程中不仅仅依赖于数据，还考虑了物理规律。PINN 依赖于物理定律的显式表达，即 PDEs 的显式形式，考虑以下初值 PDE

$$L(u)(x) = 0, \forall x \in \Omega, \quad B(u)(x) = 0, \forall x \in \partial\Omega$$

PINN 的思想即为显示的优化以下 loss。给定采样点 x_i 和边界点 y_j ，其 Loss 函数为

$$L_{loss} = \frac{1}{N} \sum_{i=1}^N |L(u)(x_i)|^2 + \lambda \cdot \frac{1}{M} \sum_{j=1}^M |B(u)(y_j)|^2$$

这里 u 参数化为神经网络函数族。借助于自动微分等技术，优化以上 loss 最后得到的函数 $u(x)$ 。

DRMs 原理类似，不过使用的是变分形式，优化最小泛函。此外还有利用方程弱解构造对抗网络训练的算法 Weak Adversarial Networks。另外 PINN 也可以用于求解反问题，本质上都是优化方程残差。

优点

- 无监督学习, 只需要采样点, 不需要 labeled 数据.
- 由于神经网络的高维非线性逼近能力, PINN 可以在高维空间中进行有效的学习, 解决一些高维 PDEs 时要比经典方法有效.
- 无网格学习, 不需要额外的网格划分, 可以处理非正则区域. 并且可以在任意网格估值(泛化性到不同网格)

缺点

- 训练过程不稳定, 黑箱, 难以解释, 理论工作尚未完全建立.
- 精度差, 低维情形无法与经典方法竞争.
- 严重依赖于物理定律的准确性和完整性, 实际上我们可能无法获取 PDEs 的精确形式, 而是观测数据, 此时无法精确构建 loss 函数.
- 训练过程不可重复, 即给一个初值就要重新训练一次. 无法应用于实时场景

1.2. Data-driven methods

数据驱动学习方法主要依赖于数据进行模型的训练和预测, 其核心思想是通过大量数据的学习, 自动提取特征和规律. 严格来说 PINN 也是数据驱动算法, 此处的 Data-driven methods 更多指向有监督学习. 由于始终我们可能只能获取到初值以及解的某些观测数据, 比如在某个网格上的估值, 而没有具体方程形式, 此时按照监督学习的范式, 可以把解 PDEs 视为函数的映射关系, 通过学习这些映射来进行预测, 即从数据中寻找 Banach 空间之间映射的逼近方法. 我们主要关注神经算子. 神经算子的相对于 PINN 等算法最大的区别是可接受不同初值条件, 可以理解为通过训练得到在函数空间的泛化性质. 考虑以下参数化 PDEs

$$L_a(u)(x) = f(x), \forall x \in \Omega, \quad B_a(u)(x) = g(x), \forall x \in \partial\Omega$$

我们可以定义解算子 $S : (a, f, g) \mapsto u$. 固定参数或边值时可以理解为将 forcing term $f \in \mathcal{F}$ 映射至解 $u \in \mathcal{M}$. 这里 $\mathcal{M}$ 在一些文献中成为解流形, 可能存在一些低维结构. 算子学习直接将此 forcing term 作为输出去拟合真实解 u . 代表的算法有 DeepONet 和 Fourier Neural Operator 以及 PCA-Net.

算子学习的一大特点是连续性学习, 即接受输入的输入为整个函数而非离散网格形式(虽然实际计算时要在有限维数据上训练). 当前研究的核心在于突破高维函数空间的表达难题. 物理模型中的输入输出常表现为连续函数(如三维温度场、随时间演化的速度场), 传统方法将其离散为高维向量会导致算法依赖特定网格, 难以适应多分辨率需求. 而算子学习具有分辨率不变的性质, 我们后面会详细介绍这一性质.

优点

- 实时响应, 即给定初值条件, 可以快速预测解. 不需要再次训练(泛化到不同函数)
- 分辨率不变: 在不同网格上训练, 可以在任意网格估值, 比如在粗网格上训练, 在细网格上估值(zero shots). 但是第一个性质不是所有算子学习算法都有, 比如 DeepONet 训练结束后无法接收不同网格的输入.
- 非线性表达: 算子学习在处理高维数据时有可能利用神经网络的高维非线性逼近能力, 从而有望克服维数灾难.

缺点

- 需要大量的 labeled 数据, 即高精度解, 这一离线过程可能需要大量的计算资源.
- 黑箱, 纯端到端模型, 可解释性差.
- 训练可能很耗时(但估值很快)

我们接下来主要关注 DeepONet 和 FNO 以及 PCA-Net 这三种神经算子, 严格来说, 所有的算子学习算法都遵循 Encoder - Decoder 架构, 我们在下面详细解释, 然后给出这三种神经算子的 Encoder - Decoder 结构.

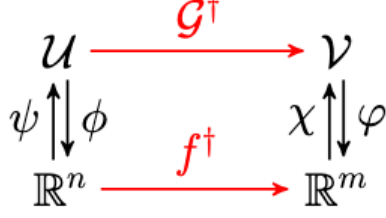
2. 基本框架

2.1. Encoder - Decoder 框架

算子学习的核心思想是将输入函数映射到输出函数, 其基本框架为 Encoder - Decoder. 但由于实际计算资源的有限, 我们的是将输入 f encode 到一个有限维空间, 然后利用神经网络非线性拟合到解的 latent space, 最后通过解码过程将其映射回输出函数 u . Encoder - Decoder 框架如下:

$$\begin{aligned}
\text{Encoder: } \mathcal{E} : f \in \mathcal{F} &\mapsto \mathcal{E}(f) \in \mathbb{R}^n \\
\text{Net: } \mathcal{N} : \mathcal{E}(f) \in \mathbb{R}^n &\mapsto \mathcal{N}(\mathcal{E}(f)) \in \mathbb{R}^m \\
\text{Decoder: } \mathcal{D} : \mathcal{N}(\mathcal{E}(f)) \in \mathbb{R}^m &\mapsto \mathcal{D}(\mathcal{N}(\mathcal{E}(f))) \in \mathcal{M}
\end{aligned}$$

这里 $\mathcal{F}$ 是输入函数空间, $\mathcal{M}$ 是输出函数空间, $\mathcal{E}$ 是编码器, $\mathcal{N}$ 是神经网络, 不过也有可能是其他形式的参数结构, $\mathcal{D}$ 是解码器. 这里的 Encoder - Decoder 框架是一个通用的框架, 具体的实现可以有不同的选择.



其次 Encoder - Decoder 一般来说会有以下性质:

$$\begin{aligned}
\phi \circ \psi &\approx \mathcal{I}, \psi \circ \phi = \mathcal{I} \\
\phi \circ \chi &\approx \mathcal{I}, \chi \circ \phi = \mathcal{I}
\end{aligned}$$

最终的神经算子可以定义为

$$\mathcal{O} = \mathcal{D} \circ \mathcal{N} \circ \mathcal{E}$$

我们的目的即利用 Data-driven 方法逼近解算子 $\mathcal{S}$.

$$\mathcal{O}(f) \approx \mathcal{S}(f)$$

2.2. 计算过程

在训练阶段, 从实验角度, 首先采样函数

$$f_i \sim \mu, \quad i = 1, 2, \dots, N$$

这里 μ 是函数空间 $\mathcal{F}$ 上的概率分布, 然后通过经典解法得到一组高精度解

$$u_i = \mathcal{S}(f_i), \quad i = 1, 2, \dots, N$$

然后通过训练得到神经算子 $\mathcal{O}$, 通过以下 loss 函数

$$L_{loss} = \frac{1}{N} \sum_{i=1}^N \|\mathcal{O}(f_i) - u_i\|^2$$

这里 $\|\cdot\|$ 是函数空间上的范数, 具体的选择可以是 L_2 范数, 实际计算中也需要在网格上取值. 通过最小化以上 loss 函数, 我们可以得到一个神经算子 $\mathcal{O}$, 使得其在训练集上的预测误差最小. 训练完成后, 我们可以通过以下方式进行预测:

$$\hat{u} = \mathcal{O}(f_{test})$$

其中 f_{test} 是新的输入函数. 通过以上方法, 我们可以有效地利用训练好的神经算子进行预测, 从而在实际应用中实现快速求解 PDEs 的目标.

2.3. 算子学习的分辨率不变性

一般来说, 要考虑两种分辨率不变性

- 输入的分辨率不变性: 即神经算子有能力接受不同分辨率的输入, 也就是说要求 Encoder 不依赖于网格. 比如提取频率(积分全局定义)
- 输出的分辨率不变性: 即神经算子可以在任意网格估值, 也就是说要求 Decoder 一定要返回函数. 这里所有神经算子都可以满足

分辨率不变性使得我们可以牺牲一些计算精度用来换取计算效率, 即我们可以在粗网格上训练神经算子, 然后在细网格上进行测试, 即 zero-shots. 实际观察发现并没有损失很多精度, 这是神经算子的一大优势. 或者说连续性算法的一大优势.

3. DeepONet

DeepONet 是一种基于深度学习的算子学习方法, 其核心思想是将 Encoder 定义为神经网络族来自适应的将输入函数 Encode 到 latent space. 该架构最初由文献Lu, Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang和George Em Karniadakis. 《Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators》. Nature Machine Intelligence 3, 期 3 (2021年3月18日): 218–29. <https://doi.org/10.1038/s42256-021-00302-5>.

提出, 作为基于Chen与Chen早期研究Tianping Chen和Hong Chen. 《Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems》. IEEE Transactions on Neural Networks 6, 期 4 (1995年7月): 911–17. <https://doi.org/10.1109/72.392253>.

的算子学习框架. 核心结构包括编码器 (encoder)、分支网络 (branch net) 和主干网络 (trunk net). 编码器将输入函数映射到低维空间, 分支网络处理编码后的输入, 主干网络生成输出函数的基函数, 最后通过线性组合得到输出.

以下是DeepONet的详细描述, 基于Kovachki等人 (2024) 的综述内容:

3.1. 架构设计

1. 输入编码 (Branch Network)

将输入函数 u 离散化为传感器点 $\{x_j\}_{j=1}^{d_U}$ 处的值 $\{u(x_j)\}$, 通过分支网络 (Branch Net) 编码为低维特征向量 $\alpha \in \mathbb{R}^P$:

$$\alpha = \text{BranchNet}(u(x_1), u(x_2), \dots, u(x_{d_U}))$$

分支网络通常为全连接网络 (MLP) 或卷积网络 (CNN)

2. 输出解码 (Trunk Network)

在输出域 $y \in \mathcal{D}_y$ 上, 通过主干网络 (Trunk Net) 生成一组基函数 $\{\psi_k(y)\}_{k=1}^{d_V}$:

$$\psi_k(y) = \text{TrunkNet}(y)_k$$

主干网络通常为MLP

3. 线性组合

将分支网络输出 α 与主干网络基函数 $\psi_k(y)$ 线性组合, 得到最终输出:

$$\Psi_{\text{DeepONet}}(u)(y) = \sum_{k=1}^{d_V} \alpha_k \cdot \psi_k(y)$$

其中 α_k 为分支网络输出的第 k 个分量, $\psi_k(y)$ 为主干网络输出的第 k 个基函数.

从架构中可以看出, 由于编码器取为神经网络函数族, 因此不具备接受不同分辨率网格的能力. 但由于 decoder 是神经网络函数, 因此还是具备不同网格估值的能力.

3.2. 理论保证

1. 通用逼近定理 (Chen & Chen, 1995)

对于任意紧集 $K \subset \mathcal{U}$ 上的连续算子 $\Psi^\dagger$, 存在 DeepONet 架构使其在 $C(K)$ 范数下任意逼近 $\Psi^\dagger$.

Theorem 4.2. Suppose that $\sigma \in C(\mathbb{R})$ is a non-polynomial activation function. Let $\mathfrak{D} \subset \mathbb{R}^d$ be a compact domain with Lipschitz boundary. Let $\Psi^\dagger : C(\mathfrak{D}) \rightarrow C(\mathfrak{D})$ be a continuous operator. Fix a compact set $K \subset C(\mathfrak{D})$. Then for any $\varepsilon > 0$, there are positive integers $d_{\mathcal{U}}, d_{\mathcal{V}}, N$, sensor points $x_1, \dots, x_{d_{\mathcal{U}}} \in \mathfrak{D}$, and coefficients $c_i^k, \xi_{ij}^k, b_i, \omega_k, \zeta_k$ with $i = 1, \dots, N, j = 1, \dots, d_{\mathcal{U}}, k = 1, \dots, d_{\mathcal{V}}$, such that

$$(4.1) \quad \sup_{u \in K} \sup_{x \in \mathfrak{D}} \left| \Psi^\dagger(u)(x) - \sum_{k=1}^{d_{\mathcal{V}}} \sum_{i=1}^N c_i^k \sigma \left(\sum_{j=1}^{d_{\mathcal{U}}} \xi_{ij}^k u(x_j) + b_i \right) \sigma(\omega_k x + \zeta_k) \right| \leq \varepsilon.$$

2. 复杂度分析 Lanthaler, Samuel, Siddhartha Mishra 和 George Em Karniadakis. 《Error estimates for DeepOnets: A deep learning framework in infinite dimensions》. arXiv, 2022年1月13日. <http://arxiv.org/abs/2102.09618>.

- 传感器点数 d_U 与输入函数的 Kolmogorov n -宽度相关。
- 逼近误差上界为：

$$\epsilon \lesssim d_U^{-\alpha} + d_V^{-\beta} + \text{NN Approximation Error}$$

其中 α, β 由输入输出空间的正则性决定。主导项是 NN error, 后面会给一个解释。

4. PCA-Net

PCA-Net 由 Bhattacharya, Kaushik, Bamdad Hosseini, Nikola B. Kovachki 和 Andrew M. Stuart. 《Model Reduction And Neural Networks For Parametric PDEs》. The SMAI Journal of Computational Mathematics 7 (2021年7月7日): 121–57. <https://doi.org/10.5802/smai-jcm.74>.

提出, 结合主成分分析 (PCA) 与神经网络

4.1. 基本框架

- 输入空间: $\mathcal{U} \subset L^2(\mathcal{D}_x)$
- 输出空间: $\mathcal{V} \subset L^2(\mathcal{D}_y)$
- 数据分布: 输入函数服从概率测度 μ , 输出为 $\Psi^\dagger(u)$ 的 forward 分布 $\Psi^\dagger_{\#}\mu$.

1. 输入编码 (PCA编码器)

- 对输入函数进行 PCA 分解, 提取前 d_U 个主成分基 $\{\phi_j\}_{j=1}^{d_U}$ 。
- 编码器将输入 u 投影到低维空间:

$$F_U(u) = (\langle u, \phi_1 \rangle, \dots, \langle u, \phi_{d_U} \rangle)^T \in \mathbb{R}^{d_U}$$

- PCA 基通过输入数据的协方差矩阵特征分解获得:

$$\text{Cov}_U = \mathbb{E}_{u \sim \mu} [u \otimes u], \quad \text{Cov}_U \phi_j = \lambda_j \phi_j$$

2. 输出解码 (PCA解码器)

- 对输出函数进行 PCA 分解, 提取前 d_V 个主成分基 $\{\psi_k\}_{k=1}^{d_V}$ 。
- 解码器将低维系数 $\beta \in \mathbb{R}^{d_V}$ 重构为输出函数:

$$G_V(\beta)(y) = \sum_{k=1}^{d_V} \beta_k \psi_k(y)$$

- 输出基通过推前分布 $\Psi_{\#}^{\dagger} \mu$ 的协方差矩阵特征分解获得.

3. 神经网络映射

- 通过神经网络 $\mathcal{N}_{\theta} : \mathbb{R}^{d_U} \rightarrow \mathbb{R}^{d_V}$ 学习输入编码到输出系数的非线性映射:

$$\beta = \mathcal{N}_{\theta}(F_U(u))$$

- 网络结构通常为MLP

完整结构:

$$\Psi_{\text{PCA-Net}}(u)(y) = \underbrace{\sum_{k=1}^{d_V} \left[\mathcal{N}_{\theta} \left((\langle u, \phi_1 \rangle, \dots, \langle u, \phi_{d_U} \rangle)^T \right) \right]_k \cdot \psi_k(y)}_{\text{解码器重构}}$$

4.2. 训练方法

1. 数据预处理

- 计算输入输出的经验协方差矩阵:

$$\text{Cov}_U^{\text{emp}} = \frac{1}{N} \sum_{i=1}^N u_i \otimes u_i, \quad \text{Cov}_V^{\text{emp}} = \frac{1}{N} \sum_{i=1}^N v_i \otimes v_i$$

- 通过奇异值分解 (SVD) 提取主成分基 $\{\phi_j\}, \{\psi_k\}$.

2. 损失函数

最小化输出函数的均方误差 (MSE):

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|G_V(\mathcal{N}_{\theta}(F_U(u_i))) - v_i\|_{L^2(\mathcal{D}_y)}^2$$

4.3. 理论分析

1. 通用逼近

Theorem 4.4 (PCA-Net universality). Let $\mathcal{U}$ and $\mathcal{V}$ be separable Hilbert spaces and let $\mu \in \mathcal{P}(\mathcal{U})$ be a probability measure on $\mathcal{U}$. Let $\Psi^{\dagger} : \mathcal{U} \rightarrow \mathcal{V}$ be a μ -measurable mapping. Assume the following moment conditions,

$$\mathbb{E}_{u \sim \mu} [\|u\|_{\mathcal{U}}^2], \quad \mathbb{E}_{u \sim \mu} [\|\Psi^{\dagger}(u)\|_{\mathcal{V}}^2] < \infty.$$

Then for any $\varepsilon > 0$, there are dimensions $d_{\mathcal{U}}, d_{\mathcal{V}}$, a requisite amount of data N , a neural network ψ depending on this data, such that the PCA-Net, $\Psi = G_{\mathcal{V}} \circ \psi \circ F_{\mathcal{U}}$, satisfies

$$\mathbb{E}_{\{u_j\} \sim \mu^{\otimes N}} [\mathbb{E}_{u \sim \mu} [\|\Psi^{\dagger}(u) - \Psi(u; \{u_j\}_{j=1}^N)\|_{\mathcal{V}}^2]] < \varepsilon,$$

where the outer expectation is with respect to the iid data samples $u_1, \dots, u_N \sim \mu$, which determine the empirical PCA encoder and reconstruction. $\diamond$

2. 误差分析

总逼近误差可分解为三部分:

$$\epsilon_{\text{total}} \leq \underbrace{\mathbb{E} [\|u - G_U(F_U(u))\|_{\mathcal{U}}^2]}_{\text{输入编码误差}} + \underbrace{\mathbb{E} [\|v - G_V(F_V(v))\|_{\mathcal{V}}^2]}_{\text{输出解码误差}} + \underbrace{\mathbb{E} [\|\mathcal{N}_{\theta}(F_U(u)) - F_V(v)\|_{\mathbb{R}^{d_V}}^2]}_{\text{映射网络误差}}$$

- 输入/输出误差由PCA截断主导，衰减率取决于特征值谱 $\lambda_j \sim j^{-\alpha}$.
- PCA编码器/解码器在均方意义下是**最优线性降维器** (Karhunen-Loève定理)。
- 映射网络逼近误差受截断误差影响.
- 一般来说无法获得 forward 测度的先验知识. (无法获得解算子的先验)

5. FNO

Li, Zongyi, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart和Anima Anandkumar.

《Fourier Neural Operator for Parametric Partial Differential Equations》. arXiv, 2021年5月16日. <http://arxiv.org/abs/2010.08895>.

由Li等人在2021年提出，旨在通过**频域积分核**高效学习无限维函数空间之间的非线性算子。其核心创新在于利用**快速傅里叶变换（FFT）**在全局范围内捕捉函数依赖关系，显著降低计算复杂度. FNO并非采用将编码/解码过程与有限维潜在空间相结合的有限维神经网络方法. 此类神经算子的本质在于将有限维神经网络结构推广至函数空间范畴，

5.1. 基本框架

输入与输出空间

- **输入空间**: $\mathcal{U} \subset L^2(\mathcal{D}_x)$
- **输出空间**: $\mathcal{V} \subset L^2(\mathcal{D}_y)$
- **核心假设**: 输入输出函数定义在**周期性域** (如 $\mathcal{D}_x = [0, 2\pi]^d$)，可通过傅里叶级数展开.
-

FNO为**层级结构**，每层包含局部线性变换和全局积分核操作：

1. 输入层（Lifting）

将输入函数 $u(x)$ 通过浅层神经网络映射到高维空间：

$$v_0(x) = P(u(x))$$

P 通常为全连接层或卷积层

2. 傅里叶层（Fourier Layers）

每层通过**局部线性变换**,**傅里叶积分核**更新特征：

$$v_{l+1}(x) = \sigma \left(W_l v_l(x) + b_l(x) + \mathcal{F}^{-1} \left(K_l(k) \cdot \mathcal{F}(v_l)(k) \right) (x) \right)$$

- $\mathcal{F}$: 傅里叶变换, $\mathcal{F}^{-1}$: 逆傅里叶变换.
- $K_l(k) \in \mathbb{C}^{d_c \times d_c}$: 可学习的频域参数（截断至低频模式 $|k| \leq k_{\max}$ ）.
- $W_l \in \mathbb{R}^{d_c \times d_c}$: 局部权重矩阵, 补偿高频信息. $b_l(x)$: 空间相关偏置项.
- σ : 激活函数算子
- 以上全是按算子意义定义.

3. 输出层（Projection）

将高维通道映射回输出空间：

$$\Psi_{\text{FNO}}(u)(y) = Q(v_L(y)) \in \mathcal{V}$$

Q 为全连接层或卷积层。

FNO通过频域参数化**非局部积分核**：

$$\mathcal{K}(v)(x) = \int_{\mathcal{D}_x} \kappa(x-y)v(y)dy \xrightarrow{\mathcal{F}} \mathcal{K}(v)(k) = \kappa(k) \cdot v(k)$$

- 截断高频：仅保留 $|k| \leq k_{\max}$

- 用残差连接类似的思想保留一些高频信息.

5.2. 训练过程

- **网格离散化**: 输入输出函数在均匀网格 $x_j = \frac{2\pi j}{N}$ ($j \in \{0, 1, \dots, N-1\}^d$) 上采样. 这里是由于 FFT 只能在均匀网格上计算, 如果是一般的网格只能用 DFT 来计算.
- **FFT加速**: 计算复杂度为 $O(N^d \log N)$, 显著优于传统积分方法 ($O(N^{2d})$).

5.3. 理论保证

1. 通用逼近定理 (Kovachki et al., 2023)

对于任意紧集上的连续算子 $\Psi^\dagger$, 存在FNO架构以任意精度逼近.

2. 复杂度优势

- **参数效率**: 频域截断使参数数量与 $k_{\max}^d$ 成比例, 而非网格点数 N^d .
- **误差衰减**: 若解的光滑性为 s , 则逼近误差满足:

$$\|\Psi_{\text{FNO}} - \Psi^\dagger\|_{L^2} \lesssim k_{\max}^{-s} + \text{NN Approximation Error}$$

6. 误差分解

在算子学习中, 通常会将总误差分解为三部分, 以便更好地理解和分析每个部分的贡献. 假设我们有一个真实的算子 $\Psi^\dagger$ 和一个近似算子 Ψ_{approx} , 我们可以定义总误差为:

$$\epsilon_{\text{total}}^2 = \mathbb{E}_{u \sim \mu} [\|\Psi^\dagger(u) - \Psi_{\text{approx}}(u)\|_{\mathcal{V}}^2] = \mathbb{E}_{u \sim \mu} [\|v - \hat{v}\|_{\mathcal{V}}^2]$$

将总误差 $v - \hat{v}$ 分解为三部分:

$$v - \hat{v} = \underbrace{(v - G_V(F_V(v)))}_{\text{Term A}} + \underbrace{(G_V(F_V(v)) - G_V(F_V(\Psi^\dagger(G_U(F_U(u)))))}_{\text{Term B}} + \underbrace{(G_V(F_V(\Psi^\dagger(G_U(F_U(u))))) - G_V(\mathcal{N}_\theta(F_U(u))))}_{\text{Term C}}$$

• Term A: $v - G_V(F_V(v))$

这直接度量了输出 v 经过输出编码-解码后的重构误差. 其 L^2 范数的平方期望定义为**解码误差** $\epsilon_{\text{decode}}^2$:

$$\epsilon_{\text{decode}}^2 = \mathbb{E} [\|v - G_V(F_V(v))\|_{\mathcal{V}}^2]$$

• Term B: $G_V(F_V(v)) - G_V(F_V(\Psi^\dagger(G_U(F_U(u)))))$

- 假设 G_V Lipschitz连续, 常数为 L_{G_V} .
- 假设 F_V Lipschitz连续, 常数为 L_{F_V} .
- 假设 $\Psi^\dagger$ Lipschitz连续, 常数为 $L_{\Psi^\dagger}$.

那么:

$$\|\text{Term B}\|_{\mathcal{V}} \leq L_{G_V} \|F_V(v) - F_V(\Psi^\dagger(G_U(F_U(u))))\|_{\mathbb{R}^{d_V}}$$

$$\leq L_{G_V} L_{F_V} \|v - \Psi^\dagger(G_U(F_U(u)))\|_{\mathcal{V}}$$

$$\leq L_{G_V} L_{F_V} L_{\Psi^\dagger} \|u - G_U(F_U(u))\|_{\mathcal{U}}$$

令 $C_1 = L_{G_V} L_{F_V} L_{\Psi^\dagger}$. 定义**编码误差** $\epsilon_{\text{encode}}^2$:

$$\epsilon_{\text{encode}}^2 = \mathbb{E} [\|u - G_U(F_U(u))\|_{\mathcal{U}}^2]$$

则 $\mathbb{E}[\|\text{Term B}\|_{\mathcal{V}}^2] \leq C_1^2 \epsilon_{\text{encode}}^2$ 。

- **Term C:** $G_V(F_V(\Psi^\dagger(G_U(F_U(u)))) - G_V(\mathcal{N}_\theta(F_U(u)))$

这一项衡量了实际的映射网络 $\mathcal{N}_\theta$ 与理想映射 $F_V \circ \Psi^\dagger \circ G_U$ 在作用于编码后的输入 $F_U(u)$ 时的差异，并通过输出解码器 G_V 体现出来。

再次使用 G_V 的Lipschitz连续性：

$$\|\text{Term C}\|_{\mathcal{V}} \leq L_{G_V} \|F_V(\Psi^\dagger(G_U(F_U(u)))) - \mathcal{N}_\theta(F_U(u))\|_{\mathbb{R}^{d_V}}$$

这个 L^2 范数的平方期望定义为**映射误差** ϵ_{map}^2 。注意，这里的映射误差衡量的是网络 $\mathcal{N}_\theta$ 逼近“理想低维映射”（即 $F_V \circ \Psi^\dagger \circ G_U$ 作用在 $F_U(u)$ 上）的能力。

$$\epsilon_{\text{map}}^2 = \mathbb{E} [\|F_V(\Psi^\dagger(G_U(F_U(u)))) - \mathcal{N}_\theta(F_U(u))\|_{\mathbb{R}^{d_V}}^2]$$

则 $\mathbb{E}[\|\text{Term C}\|_{\mathcal{V}}^2] \leq L_{G_V}^2 \epsilon_{\text{map}}^2$ 。

3. 合并误差项

$$\|v - \hat{v}\|_{\mathcal{V}}^2 \leq 3 (\|\text{Term A}\|_{\mathcal{V}}^2 + \|\text{Term B}\|_{\mathcal{V}}^2 + \|\text{Term C}\|_{\mathcal{V}}^2)$$

对上式取期望：

$$\mathbb{E}[\|v - \hat{v}\|_{\mathcal{V}}^2] \leq 3 (\mathbb{E}[\|\text{Term A}\|_{\mathcal{V}}^2] + \mathbb{E}[\|\text{Term B}\|_{\mathcal{V}}^2] + \mathbb{E}[\|\text{Term C}\|_{\mathcal{V}}^2])$$

即

$$\epsilon_{\text{total}}^2 \leq 3 (\epsilon_{\text{decode}}^2 + C_1^2 \epsilon_{\text{encode}}^2 + L_{G_V}^2 \epsilon_{\text{map}}^2)$$

- **编码误差:** 度量了将输入函数 u 通过编码器 $F_U : \mathcal{U} \rightarrow \mathbb{R}^{d_u}$ 映射到潜在空间，再通过（概念上的）输入解码器 $G_U : \mathbb{R}^{d_u} \rightarrow \mathcal{U}$ 重构回原始空间时引入的误差。本质上是**输入空间表示误差**，即用 d_u 维向量表示无限维输入函数所造成的精度损失。
 - **依赖:** 输入函数空间的性质（如光滑度、PCA谱衰减率）、编码器/解码器的选择、以及潜在空间维度 d_u 。
- **重构/解码误差:** 度量了将输出函数 v 通过（概念上的）输出编码器 $F_V : \mathcal{V} \rightarrow \mathbb{R}^{d_v}$ 映射到潜在空间，再通过输出解码器 $G_V : \mathbb{R}^{d_v} \rightarrow \mathcal{V}$ 重构回原始空间时引入的误差。本质上是**输出空间表示误差**，即用 d_v 维向量表示无限维输出函数所造成的精度损失。
 - **依赖:** 输出函数空间的性质（真实算子 $\Psi^\dagger$ 的值域特性）、编码器/解码器的选择、以及潜在空间维度 d_v 。
- **神经网络逼近误差:** 度量了神经网络 $\psi : \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_v}$ 逼近**真实算子在潜在空间中的映射** $\phi = F_V \circ \Psi^\dagger \circ G_U$ 的误差。这是有限维函数逼近问题。这一项相对来说是主导项。

我们下面的关于神经网络逼近函数的速率 bound.

Theorem 5.2. A function $f \in W^{k,\infty}([0,1]^d)$ can be approximated to uniform accuracy $\epsilon > 0$,

$$\sup_{x \in [0,1]^d} |f(x) - \psi(x)| \leq \epsilon,$$

by a ReLU neural network ψ with at most $O(\epsilon^{-d/k} \log(\epsilon^{-1}))$ tunable parameters.

◇

这里的速率受到维数影响. 在算子学习中, 由于此维数在输入端被放大 $\mathcal{O}(1) \rightarrow \mathcal{O}(\epsilon^{-\alpha})$. 所以算子学习受维数诅咒要比有限维更严重, 本质来源于无穷维空间的复杂度要比有限维大.